

Introduction To Spring WebFlux

How does the Event Loop Work?

1. **Single Thread, Continuous Loop:** One thread keeps running in a tight loop.

2. **Checks for Events like:**

- New incoming HTTP requests
- Responses from databases or external APIs

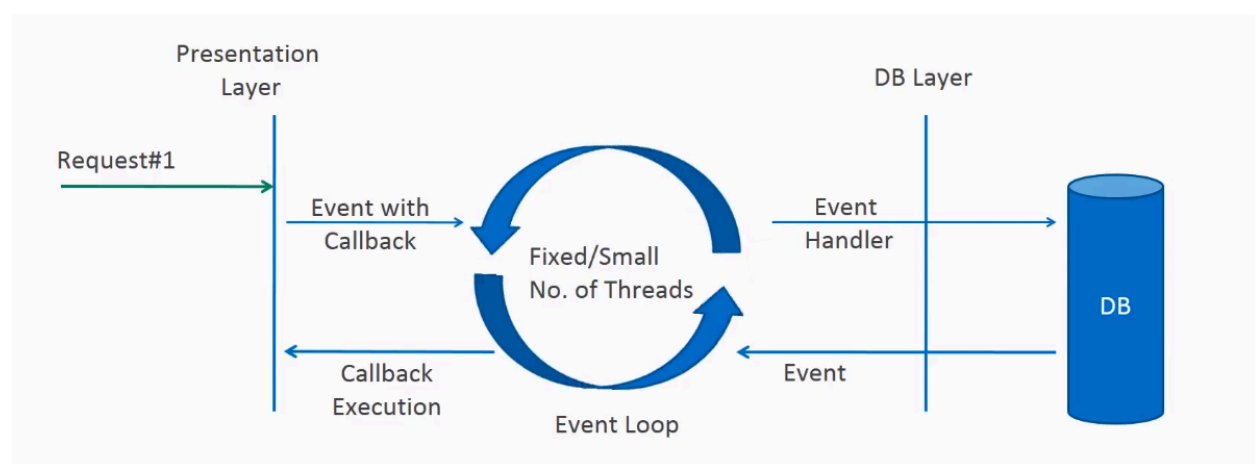
3. **Non-Blocking I/O**

When an I/O operation is needed (e.g., calling a database), the thread **does not wait**. Instead, it registers the operation and moves on to handle the next event.

4. **Callback / Notification**

When the I/O operation completes, the Event Loop gets notified and processes the result.

This way, one thread can manage **thousands of connections** at the same time.



- ⚡ This single-thread event loop model is one valid solution to the thread blocking problem. Virtual threads solve the exact same problem differently, instead of one thread juggling thousands of connections via callbacks, the JVM gives every request its own thread but makes that thread nearly free to create and cheap to park while waiting.

What is Spring WebFlux?

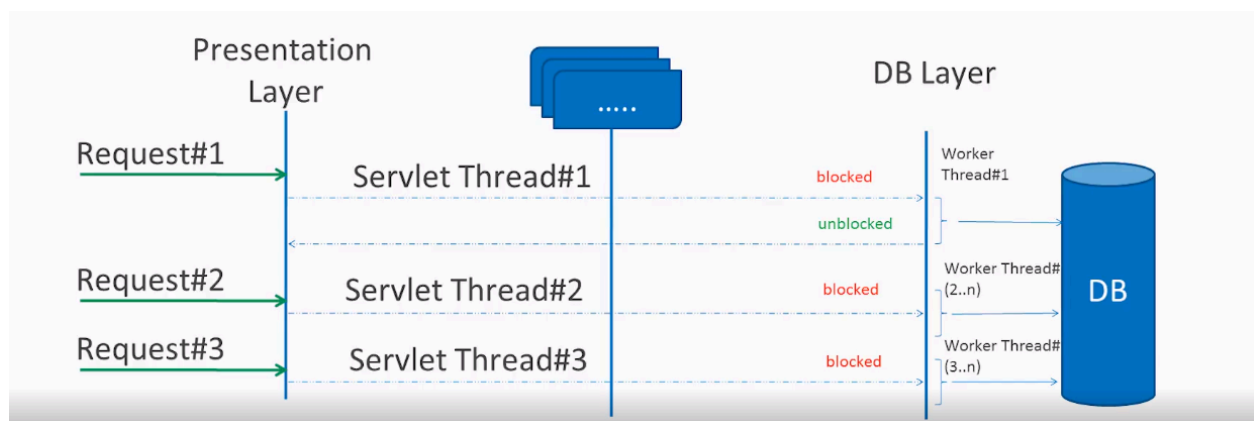
Spring WebFlux is Spring Framework's **reactive web framework**. It is fully **non-blocking**, **asynchronous**, and built on the **Reactive Streams** specification.

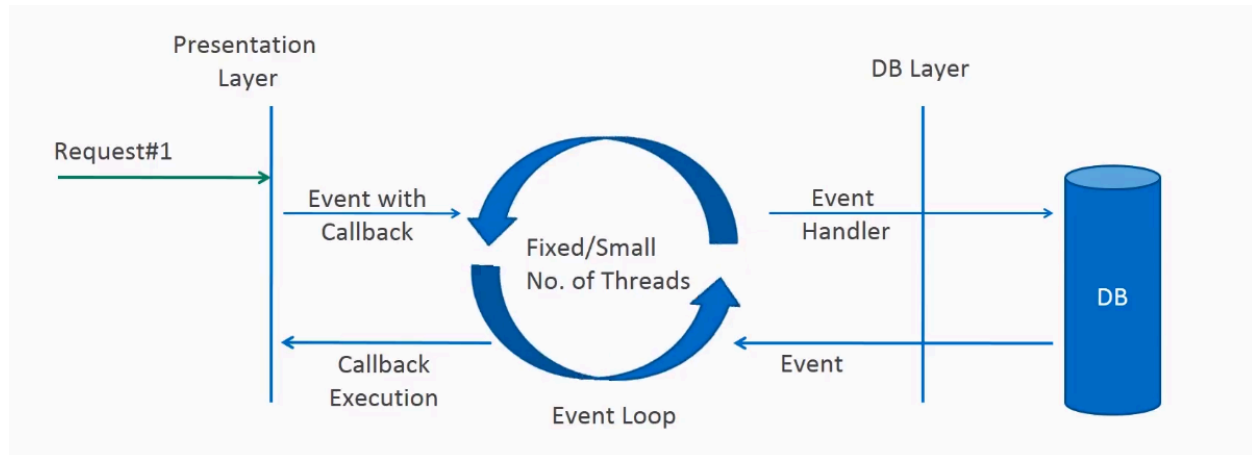
It allows you to build highly scalable, event-driven web applications using **Project Reactor** (**Flux** and **Mono**) as its core reactive library.

Why Spring WebFlux?

Assigning one waiter to each guest coming into the hotel vs one waiter taking request from all the guests as they come in.

- ⚡ Virtual threads are like giving every guest their own hologram waiter, which are cheap to make that having 10,000 of them is no longer a problem. The event loop model says "one waiter serving everyone is more efficient." Virtual threads say "just make waiters basically free." Both solve the original staffing crisis, through opposite philosophies.





In Spring MVC, Tomcat gives each request a thread. The default pool is 200. Request 201 waits. Under sustained load, the waiting queue grows, memory spikes, and the server starts rejecting requests. Not because the CPU is busy, but because threads are all parked waiting for DB responses.

WebFlux flips the model entirely. Instead of assigning a thread per request, it uses an **event loop**, a small fixed set of threads that never block. When a request needs to wait for I/O, the thread doesn't wait with it. It moves on to the next request. A callback fires when the data is ready.

Core Features of Spring WebFlux

Feature	Spring MVC (Blocking)	Spring WebFlux (Reactive)
Threading Model	One thread per request	Few threads + non-blocking
Programming Style	Imperative	Reactive + Declarative
Server	Tomcat, Jetty (Servlet)	Netty (default), Undertow
Best For	Simple CRUD apps	High concurrency, streaming
Backpressure	Not supported	Built-in



As of Java 21 with virtual threads enabled, the "Threading Model" row for Spring MVC changes from "one OS thread per request, expensive" to "one virtual thread per request, cheap."

The "Best For" row also shifts, simple CRUD apps still favor MVC, but high concurrency is no longer automatically a WebFlux-only strength. Streaming with backpressure remains the one row where WebFlux still wins clearly, covered later in this module.

How does the Netty Server work internally

Phase 1: Connection & Request Ingestion

1. Client Connects

The client's browser knocks on the server's door by sending a tiny network packet called a "TCP SYN" across the internet.

2. Connection Notification

The OS Kernel uses the non-blocking `epoll` mechanism to notify Netty's **Boss Thread** that a new connection is ready to be handled. The Kernel creates a brand new **Socket** (a dedicated mailbox) for this specific client.

3. Registering the Channel

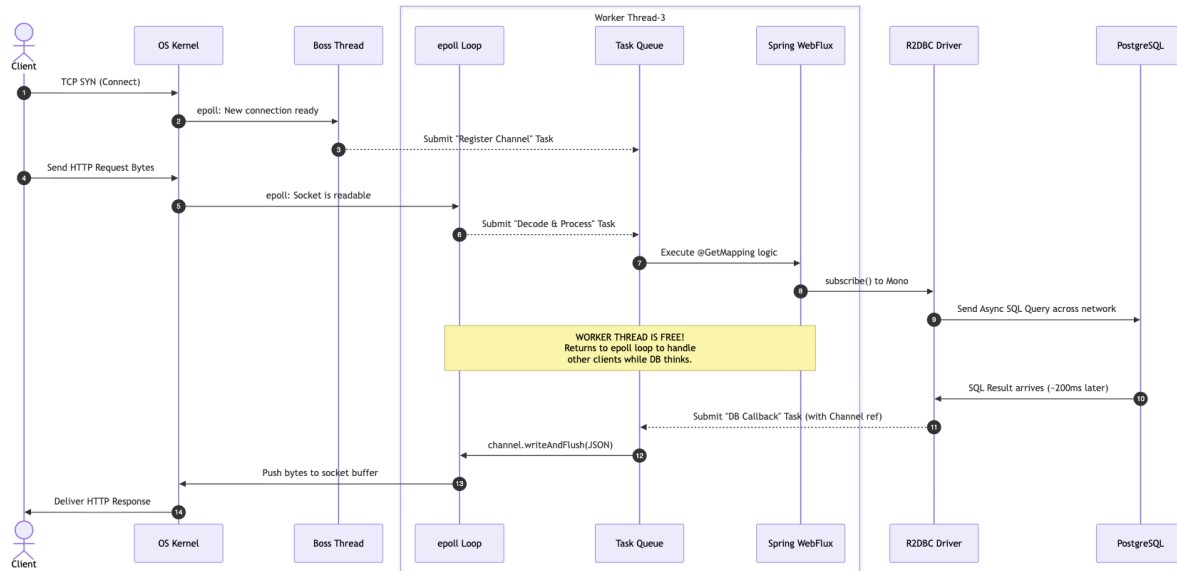
The Boss Thread doesn't actually serve the customer; its only job is to greet them. It takes the reference to this new connection (the Channel) and drops a note into a **Worker Thread's Task Queue**.

4. Request Data Sent

The client actually sends the HTTP request data (e.g., `GET /users/42`). The client sends the actual HTTP request bytes over the established network connection to the OS Kernel.

5. Socket Read Trigger

The mailbox is no longer empty. The **OS Kernel** immediately fires an `epoll` notification to the specific Worker Thread managing this channel.



Phase 2: Processing & Reactive I/O Handoff

6. Worker Routes Task

The Worker Thread's `epoll` loop reads the raw bytes out of the socket. It then wraps those bytes into a "Decode & Process" task and pushes it into its own internal **Task Queue**.

7. Execute `@GetMapping` logic

The Worker Thread processes the task. It parses the raw HTTP bytes into an official HTTP Request object and routes it straight into your **Spring WebFlux** controller logic.

8. Database Subscription

Your code calls the database. Because this is reactive (Spring WebFlux), it does **not** block the thread waiting for a response. Instead, it calls `.subscribe()` on a reactive stream via the **R2DBC Driver**.

9. Async Query Dispatched

The R2DBC Driver takes the SQL query, opens a network connection to PostgreSQL, and fires the query packets across the wire.



At this exact moment, the Netty **Worker Thread is completely freed**. Instead of waiting idling for the database, it immediately returns to its `epoll Loop` to process incoming traffic from other clients.

Phase 3: Database Response & Client Delivery

10. Database Responds

After processing (~200ms later), PostgreSQL finishes finding User 42. It sends the database rows back across the internal network.

11. Database Callback Triggered

The R2DBC driver wakes up, grabs the data, and looks at that `.subscribe()` left in Step 8. It knows this data belongs to the original client channel. It submits a "DB Callback" task into the Netty Worker's **Task Queue**.

12. Preparing the Response

The Worker Thread picks up the callback task from its queue. It passes the database data through Spring to turn it into JSON bytes, and invokes Netty's `channel.writeAndFlush()`.

13. Flushing to OS Buffer

Java pushes those JSON bytes out of its own JVM memory and **flushes** them into the **OS Kernel's** network socket buffer.

14. Final Delivery

The **OS Kernel** takes control of the physical hardware. It chops that JSON payload into network packets and streams them across the physical internet cables back to the **Client**.